Task 1: Evaluate the Requirements of Each Computing Environment

Bloom level: K5 Evaluate

Aim

Find out what each environment needs from its operating system, rank those needs, and show which ones conflict.

Explanation

A requirement is a property the OS must provide. The main ones are:

- **Response time:** how fast the system reacts to an event (for example, a sensor input). Hard deadlines matter most for monitors.
- **Reliability:** how long the system runs without failure, and how it recovers.
- **Throughput:** how much work is finished per second (records per second, images per minute).
- **Battery:** energy used per hour. Only matters for battery devices (tablets).
- **Security:** protection of patient data and devices from misuse.

Ranked requirements for each environment

Rank 1 is the most important. The reason is given in the last column.

Env	Rank 1	Rank 2	Rank 3	Rank 4	Rank 5	Justification of ranking
E1 Monitor	Deadline (alarm within bound)	Reliability	Response-time predictability	Security	Throughput	A late alarm can harm a patient. Monitors do very little work, so throughput is last.
E2 Tablet	Reliability (full shift)	Security (lost device)	Battery	Response time (UI)	Throughput	Nurses rely on the device for a whole shift. Lost devices expose records. UI delay is annoying but not dangerous.
E3 Workstation	Security (record access)	Response time (UI)	Reliability	Throughput	Battery (not used)	Doctors open many records. Access control is the top legal and ethical duty.
E4 DB server	Reliability / availability	Integrity (no lost writes)	Security	Throughput (transactions/s)	Response time (query)	If records are unavailable, care stops. Integrity prevents wrong data.

Env	Rank 1	Rank 2	Rank 3	Rank 4	Rank 5	Justification of ranking
E5 Imaging server	Throughput (reconstruction)	Reliability (no lost images)	Security	Response time	Battery (not used)	Image work is long and batch-like. Lost data is costly, but speed is the main goal.
E6 Partner link	Security (encryption, firewall)	Reliability (retry)	Throughput	Latency	Battery	The link is the entry point for attacks. Data must arrive correctly, not just quickly.
E7 MRI controller	Hard real-time timing	Reliability	Throughput (data bandwidth)	Security	Battery	Gradient pulses must be exact. A timing error can spoil an image or harm equipment.

Conflicts between requirements

Conflict	Example	How it is resolved
Fast response vs many users	Many doctors on one server slow each other down	Separate machines per role; priority scheduling for interactive work
Throughput vs deadline	Batch image jobs delay a monitor alarm	Never run batch work on monitors; use a real-time OS with fixed priorities
Security vs response time	Encryption and logging add latency	Hardware encryption (AES instructions); encrypt only what leaves the device
Battery vs performance	High CPU clock drains the tablet	Dynamic frequency scaling; reduced background services
Reliability vs cost	Fail-over needs spare machines	Clusters only for the database; single machines elsewhere

Requirements critical to patient safety

- **Alarm delivery on monitors:** the alarm must be raised before a fixed deadline, and it must not be silently lost.
- **Continuous access to records:** a doctor must be able to read medication and allergy data during care.
- **Correct data:** a wrong blood-group or dosage value is a safety failure even if the system is fast.
- **Gradient timing on the MRI:** incorrect timing can distort images or create hazards.

How OS history applies to each environment

OS era	Key idea	Where it applies here
Batch (1950s to 60s)	Jobs run one after another without interaction	Overnight reconstruction and backups on E5 and E4

OS era	Key idea	Where it applies here
Time-sharing (1960s to 70s)	Many users share one machine with quick switching	Workstations (E3) and the database server (E4) serving many clients
Personal computing (1980s to 90s)	One user, rich interface, single-user protection	Doctor workstation UI design; lessons on why protection still matters
Mobile (2000s onward)	Battery, touch, app sandboxes	Nurse tablets (E2)
Real-time systems (1970s onward)	Guaranteed deadlines with priority scheduling	Bedside monitors (E1) and MRI controller (E7)

Alternative concept: one general-purpose OS on every device

If every device ran one general-purpose OS (for example, standard Linux or Windows), the problems below appear. The analysis uses Big-O for scheduling and memory size per device type.

Measure	Dedicated OS per environment	Single general-purpose OS everywhere
Worst-case alarm response time	Bounded by a fixed deadline (for example, a few milliseconds) with a real-time OS. Scheduling decision cost O(1) or O(log n) in the number of ready tasks.	Unbounded: standard fair scheduling (for example, CFS in Linux, O(log n) per decision) gives no hard guarantee. A busy system can delay an alarm by tens of milliseconds or more.
Memory per device (typical)	Real-time kernel: tens of KB to a few MB. Tablet OS: hundreds of MB.	General OS base: often several hundred MB to GB on every monitor. Monitors have much less RAM, so this may not fit.
Administration effort	Several images, each small and fixed. Patches tested per role.	One image, so patches are simpler, but one bad update can disable monitors and workstations together. Testing is harder.
Result	Fits the safety requirements	Fails the deadline requirement on monitors, but is acceptable for workstations.

Result for Task 1. Ranked requirement tables are given for seven environments. Safety-critical requirements are deadlines (monitors, MRI), continuous reliable access (database, tablets) and protection of records (workstations, partner link). A single general-purpose OS is rejected for monitors because it cannot guarantee the alarm deadline, but it is acceptable for workstations.

Task 2: Judge the Type of Operating System for Each Environment

Bloom level: K5 Evaluate

Aim

Compare the main OS types and select the best type for each environment, with reasons for rejecting the others.

Comparison of OS types

OS type	Main feature	Strength	Weakness
Mainframe	Batch and transaction processing on large central machines	Very high reliability and I/O throughput	Expensive; poor fit for interactive devices
Desktop	One user, rich graphical interface	Good interaction	No hard deadlines; no fail-over
Multiprocessor	Several CPUs in one machine	Parallel work speeds up large jobs	One machine can still fail
Distributed	Several machines appear as one system	Resource sharing, scaling	Network delays; harder to manage
Clustered	Several machines with shared storage and fail-over	High availability	Complex; needs shared storage and heartbeats
Real-time (hard)	Deadlines must always be met	Predictable timing	Limited features; certification effort
Real-time (soft)	Deadlines should usually be met	Good media and video timing	Late results are tolerated
Handheld	Touch, low power, apps	Battery life, touch input	Limited memory and storage

Best type for each environment

Environment	Chosen type	Rejected alternatives and why
Bedside monitors (E1)	Real-time, hard deadlines	Desktop: no guarantee. Soft real-time: a late alarm is not acceptable.
Nurse tablets (E2)	Handheld	Desktop: drains battery, too heavy. Hard real-time: not needed for UI.

Environment	Chosen type	Rejected alternatives and why
Doctor workstations (E3)	Desktop (time-sharing, multiuser protection)	Handheld: too few features. Real-time: not required.
Database server (E4)	Clustered with fail-over (on top of a multiprocessor)	Single multiprocessor: one failure stops records. Mainframe: costly and unnecessary.
Imaging server (E5)	Multiprocessor, symmetric (SMP)	Asymmetric: see below. Cluster: image data would need to be moved between nodes.
Partner link (E6)	Distributed-network appliance (hardened, general-purpose embedded Linux)	Desktop: too exposed. Real-time: not needed.
MRI controller (E7)	Real-time, hard deadlines	Desktop or general Linux: unpredictable timing (see Task 1).

Symmetric versus asymmetric multiprocessing for the imaging server

Feature	Symmetric (SMP)	Asymmetric (AMP)
How work is assigned	Every CPU runs the OS and any task. Scheduler picks work from one shared ready queue.	One master CPU runs the OS and assigns work to the others.
Load balance	Good. Cost of scheduling is O(p) per tick for p CPUs.	Poor when the master is busy; the master is a bottleneck.
Failure	If one CPU fails, the rest keep working (with care).	If the master fails, the whole machine fails.
Speed-up (Amdahl)	S = 1 / ((1 - f) + f / p). Near-linear for a parallel job where f is close to 1.	Limited by the master: its share of the work is a serial part, so S is smaller.
Decision	Chosen. Image reconstruction is parallel and must not be bottlenecked.	Rejected.

Worked example (illustrative): if 90 percent of reconstruction (f = 0.9) runs in parallel on p = 8 CPUs, S = 1 / (0.1 + 0.9 / 8) = 1 / 0.2125 = about 4.7 times faster. With p = 16, S = 1 / (0.1 + 0.05625) = about 6.4. The serial 10 percent limits the gain, so the design should minimise serial steps.

Clustered system with fail-over for the database server

Two or more database nodes share access to the storage. A heartbeat is sent between nodes at a fixed interval h (for example, every 1 second). If the standby misses k heartbeats (for example, 3), it takes over the shared storage and starts the database service. Typical fail-over takes a few seconds, which is far better than hours of downtime for a restore from backup.

Design note. Fail-over needs fencing (STONITH or similar) so that a node that is only slow cannot write to the shared disks at the same time as the new primary. Without fencing, records can be corrupted.

Alternative: one large multiprocessor machine for the database

Measure	Cluster with fail-over	Single large multiprocessor
Availability	High. Fail-over in seconds after a node fails. Availability depends on the detection timeout.	Single point of failure. A hardware fault stops all records until repair. Availability is lower.
Cost	Several medium servers plus shared storage. Hardware cost grows linearly with nodes.	One very large machine has a high price premium per CPU and per GB at the top end.
Throughput vs processors	Scales across nodes but needs coordination. Overall cost of a transaction is O(1) for local work, plus network cost O(k) for k remote nodes.	Scales with p processors according to Amdahl; lock contention grows with p, so gains flatten.
Memory available (space)	Each node has its own RAM. Cache is per node, so hot data can be duplicated, costing O(n x nodes) space.	One large pool of RAM, so caching is simpler and total memory is O(M) with no duplication.
Decision	Chosen, because availability is the top requirement (Task 1).	Rejected as the only database machine. Acceptable as a node inside the cluster.

Result for Task 2. Monitors: hard real-time. Tablets: handheld. Workstations: desktop. Database: clustered with fail-over over multiprocessor nodes. Imaging: SMP. Partner link: hardened embedded Linux appliance. MRI controller: hard real-time. Rejected options are listed above with reasons.

Task 3: Evaluate Operating-System Structures

Bloom level: K5 Evaluate

Aim

Compare the main OS structures and choose a structure for the monitors and for the database server.

Structures compared

Structure	How it is built	Reliability	Performance	Ease of updating
Simple (monolithic)	All OS services in one big kernel address space (for example, early MS-DOS, original UNIX)	Low: one bug can crash all	High: a service call is one function call inside the kernel	Hard: changing one part means rebuilding the kernel
Layered	Kernel split into layers; each layer uses only the one below	Medium: layers isolate some faults	Lower: extra layer crossings	Medium: layers can be replaced if the interface is kept
Microkernel	Only IPC, basic scheduling and memory are in the kernel; drivers and file systems run as user servers	High: a crashed server can be restarted	Lower: each service call uses message passing	Good: servers replaced without reboot

Structure	How it is built	Reliability	Performance	Ease of updating
			(two context switches)	
Modular (loadable modules)	Kernel core plus modules loaded at run time (for example, Linux LKMs)	Medium to high: modules share the kernel address space	High: direct calls	Good: load or unload without reboot
Hybrid	Mixture of monolithic speed with microkernel-style structure (for example, Windows NT executive, macOS XNU)	Medium to high	High	Good

How common OSes are structured

- **Linux:** monolithic kernel with loadable kernel modules (modular). Device drivers run in the kernel.
- **Windows:** hybrid. An executive with layers runs in kernel mode; some services run in user mode.
- **Android:** layered. Linux kernel at the bottom, then a hardware abstraction layer (HAL), native libraries, the Android runtime, and apps in sandboxes.
- **FreeRTOS:** a small kernel that provides tasks, queues and timers. It is compact and structured like a simple kernel with optional modules. It is not a full OS.
- **QNX:** microkernel. Drivers and file systems run as separate processes that talk by message passing. Designed for hard real-time and high reliability.

Which structure suits each device

System	Structure chosen	Reason
Patient monitors (E1)	Microkernel (QNX-style) or a compact RTOS kernel	A faulty driver (for example, the ADC sensor driver) should restart without stopping alarms. Microkernel isolates faults. A compact RTOS keeps memory small.
Database server (E4)	Modular monolithic kernel (Linux with loadable modules)	Database performance needs fast system calls and I/O paths. Modules let storage and network drivers be replaced without reboot. Strong community testing lowers risk.
Nurse tablets (E2)	Layered (Android-style)	Sandboxes per app, hardware abstraction helps many device models.
Imaging server (E5)	Modular monolithic kernel	Fast I/O and driver support for GPUs and storage.

Alternative: monolithic kernel instead of microkernel on the bedside monitor

Measure	Microkernel	Monolithic kernel
Time cost of a service request	Message passing between client and server: about two context switches plus	One system call (trap) and a function call: O(1) in kernel space. Usually faster.

Measure	Microkernel	Monolithic kernel
	copies. Cost O(k) for k servers in the path. Example: a sensor read may cross 2 servers, so 4 switches.	
Memory taken by kernel	Small: tens of KB to a few hundred KB resident kernel.	Larger: usually 1 MB or more, because drivers live in the kernel. Monitors have little RAM, so this matters.
Effect of a faulty driver	Only the driver process crashes. It is restarted; alarm path continues.	A bug in the driver can corrupt memory or crash the whole monitor, so alarms can stop.

Trade-off: a monolithic kernel is faster per call, but the microkernel protects the alarm path better. For a monitor, safety is ranked above speed (Task 1), so the microkernel or compact RTOS is preferred.

Result for Task 3. Monitors use a microkernel or compact RTOS structure for fault isolation and small memory. The database server uses a modular monolithic kernel for speed and easy driver updates. Both choices are defended with the trade-offs above.

Task 4: Evaluate the Services an Operating System Must Provide

Bloom level: K5 Evaluate

Aim

Decide which OS services each environment needs, remove the ones it does not need, and explain how the OS protects patient records.

Service table

Key: Y = required, N = removed, O = optional, E = essential for safety.

Service	Monitor (E1)	Tablet (E2)	Workstation (E3)	DB server (E4)	Imaging (E5)
User interface	N (small display only)	Y (touch UI)	Y (full GUI)	N (remote admin only)	O (viewer)
Program execution	Y (fixed firmware tasks)	Y	Y	Y	Y
I/O operations	E (sensors, display)	Y	Y	E (disks, network)	E (scanner, disks)
File systems	N (log to flash only, simple)	Y	Y	E	Y
Communication	O (to central station)	Y	Y	E (client access)	Y

Service	Monitor (E1)	Tablet (E2)	Workstation (E3)	DB server (E4)	Imaging (E5)
Error detection	E (sensor faults, watchdog)	Y	Y	E (logs, checksums)	Y
Resource allocation	Y (fixed)	Y	Y	E	E
Accounting	N	O	O	Y (audit)	O
Protection	E (task isolation)	Y (app sandbox)	Y	E (record isolation)	Y
Security	E (tamper, signed firmware)	E (device loss, encryption)	E (login, audit)	E (encryption, access control)	Y

Services removed from the bedside monitor to make it smaller and faster

- Graphical windowing system: replaced by a fixed screen driver.
- User accounts and shell: the monitor runs one fixed firmware image.
- Accounting and general file system: logs are written to a fixed-size circular buffer in flash.
- Dynamic program loading and unrestricted process creation: all tasks are fixed at build time.

Removing these lowers memory use and shortens the path for an alarm, but the safety services (error detection, watchdog, task isolation) must stay.

Services essential for the database server

Resource allocation (CPU, memory and disk shares for many clients), file and storage services with journaling, communication (secure client access and replication), protection (per-user and per-record permissions), security (encryption and audit logs), and error detection (checksums and fail-over). Accounting is required for audit trails.

How the OS protects one patient record from another user program

Mechanism	What it does	Example
Hardware mode bit and privileged instructions	User programs cannot change page tables or disable interrupts	A user program cannot disable the memory check
Virtual memory per process (MMU)	Each process sees only its own address space	Program A cannot read the memory of the DB process
File permissions and access control lists	Only allowed users can open a record file	A nurse cannot open a file that is not assigned to her
System-call checks	The kernel validates every pointer and file descriptor it receives	A bad buffer pointer returns an error (EFAULT), not a crash
Authentication and audit logs	Users prove identity; each access is logged	Every read of a record is traceable
Encryption at rest and in transit	Stored and sent data are unreadable without the key	A stolen disk does not reveal records

Alternative: full desktop OS on tablets instead of a reduced handheld OS

Measure	Reduced handheld OS	Full desktop OS with all services
Memory used by extra services (space)	Small: the base OS and UI stack take a few hundred MB at most	Extra daemons, indexing and printing add hundreds of MB. Space cost is O(s) for s services enabled.
Storage used	A few GB for the image	Tens of GB for a full install and updates
Start-up time (time)	Shorter; boot work is roughly O(s) with s services, and s is small	Longer: every service adds start-up work, so boot time is O(s) with a large s
Battery life (time)	Longer: fewer background wake-ups	Shorter: background services wake the CPU and radios more often

Result for Task 4. Each environment has a service table with inclusions and exclusions. The monitor drops the windowing system, general file system and accounting. The database server keeps protection, accounting, file and communication services. Records are protected by the mode bit, per-process memory, file permissions, checked system calls, audit logs and encryption.

Task 5: Examine System Calls

Bloom level: K5 Evaluate

Aim

Explain how a program asks the OS for a service, step by step, and show the evidence with strace.

Categories of system calls

Category	Purpose	Typical Linux calls
Process control	Create, end, wait for and inspect processes	fork, execve, exit, wait4, clone
File management	Open, read, write, close and inspect files	open, read, write, close, lseek, stat
Device management	Request and release devices, control them	ioctl, read/write on /dev files
Information maintenance	Get or set system and process information	getpid, time, clock_gettime, getrusage
Communication	Send data between processes or machines	pipe, socket, connect, send, recv
Protection	Change permissions and owners	chmod, chown, setuid, umask

What happens when a program calls read()

Example: ssize_t n = read(fd, buf, 4096);

Step	What happens	Where
1. Library call	The program calls read(). In glibc this is a small wrapper function. It puts the arguments into CPU registers.	User mode, C library
2. Trap	The wrapper puts the system call number for read (0 on x86-64) in register rax and executes the syscall instruction. This is a trap: it jumps into the kernel.	Hardware

Step	What happens	Where
3. Mode switch	The CPU switches from user mode to kernel mode and jumps to the kernel entry point. The user registers are saved on the kernel stack.	CPU
4. Kernel work	The kernel checks the file descriptor, finds the file in the per-process table, and calls the file system. If the data is not in the page cache, it is read from disk (this may sleep and let other tasks run). Data is copied into user memory with a checked copy routine.	Kernel mode
5. Return	The number of bytes read (or a negative error) is returned in rax. The kernel restores user registers and returns to user mode. On error, the wrapper sets errno.	Kernel to user mode

How parameters are passed to the kernel

- **Registers (used by Linux x86-64):** up to six arguments go in rdi, rsi, rdx, r10, r8 and r9. This is the fastest method.
- **A block in memory:** for large or structured data, the program puts a structure in memory and passes its address in a register. Example: stat() fills a struct stat buffer.
- **The stack:** used when there are more arguments than registers. Many 32-bit systems and some other OSes use it. Linux x86-64 uses registers for all normal calls.

How strace shows system calls

strace is a tool that traces the system calls made by a program. It shows each call, its arguments, its return value and the time taken. Useful commands:

```
strace ./copy_rw test.bin out.bin 4096          # show every call
strace -c ./copy_rw test.bin out.bin 4096       # summary table of call counts
strace -e trace=read,write ./copy_rw ...        # only read and write
strace -T -e trace=read ./copy_rw ...           # time spent in each read
```

Sample output (ILLUSTRATIVE, not a measurement. Replace it with the output from your own run):

```
openat(AT_FDCWD, "test.bin", O_RDONLY) = 3
openat(AT_FDCWD, "out.bin", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 4
read(3, "\x8f\x1a...", 4096) = 4096
write(4, "\x8f\x1a...", 4096) = 4096
read(3, "\x2c\xb7...", 4096) = 4096
...
read(3, "", 4096) = 0
close(3) = 0
close(4) = 0
```

Evidence to include

- Your own strace -c output for one copy run (paste it in the report).
- A short explanation of one read line: the arguments, the return value and what it means

Result for Task 5. System calls are grouped into six categories. A read() call moves through a library wrapper, a trap, a mode switch, kernel work and a return. Arguments travel in registers, memory blocks or the stack. strace makes each call visible

Task 6: Evaluate the Cost of System Calls

Bloom level: K5 Evaluate

Aim

Measure how the buffer size changes the number of read() and write() calls when a file is copied, and recommend a buffer size for the hospital backup program.

Why a system call costs much more than a function call

An ordinary function call stays in user mode and costs a few nanoseconds. A system call needs a trap, a mode switch to kernel mode, checks of arguments, possibly a lock, and a switch back. On modern CPUs this is typically hundreds of nanoseconds to a few microseconds, and mitigations for CPU security issues can add more. Each call also disturbs the CPU caches.

How buffer size changes the number of calls

If the file has n bytes and the buffer has b bytes, a copy needs about n / b reads and about n / b writes, plus one final read that returns 0 at end of file. Each call copies up to b bytes, so the total copy work is O(n) no matter what b is. Only the number of calls (and their overhead) changes.

Calculated (not measured) read counts for a 1 MB file

These are computed with n / b. They are arithmetic, not measurements. The 1 MB file is 1,048,576 bytes.

Buffer size b (bytes)	Data reads = n / b	Total read() calls (with final EOF read)	Write() calls (same as data reads)
1	1,048,576	1,048,577	1,048,576
64	16,384	16,385	16,384
512	2,048	2,049	2,048
4,096	256	257	256
65,536	16	17	16
1,048,576	1	2	1

How library buffering (fread, fwrite) reduces system calls

The C library (stdio) keeps its own buffer, usually 4 KB (the file system block size). Calling fread() for one byte takes the byte from the library buffer. A system call happens only when the library buffer is empty or full. So the number of system calls is about n / 4096 for a sequential file, even if the program asks for one byte at a time. The cost is extra memory for the library buffer and one extra copy from kernel to library buffer to user buffer.

Aim of the code: copy program with read() and write()

The program below copies a file using read() and write() with a buffer of size given on the command line. It counts how many read() and write() calls it made, so the count can be checked against strace -c.

Code: Task06 (TODO 1 completed)

```
/* copy_rw.c - copy a file with read() and write() using a chosen buffer size.
 * Compile: gcc -O2 -Wall -o copy_rw copy_rw.c
 * Run:     ./copy_rw <source> <destination> <buffer_size_in_bytes>
 */
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>

int main(int argc, char *argv[])
{
    if (argc != 4) {
        fprintf(stderr, "Usage: %s <source> <destination> <buffer_size>\n", argv[0]);
        return 1;
    }

    const char *src = argv[1];
    const char *dst = argv[2];
    long bufsize = atol(argv[3]);
    if (bufsize <= 0) {
        fprintf(stderr, "buffer size must be greater than 0\n");
        return 1;
    }

    char *buf = malloc((size_t)bufsize);
    if (buf == NULL) {
        perror("malloc");
        return 1;
    }

    int in = open(src, O_RDONLY);
    if (in < 0) {
        perror("open source");
        free(buf);
        return 1;
    }

    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) {
        perror("open destination");
        close(in);
        free(buf);
        return 1;
    }

    long long total = 0;      /* bytes copied            */
    long long read_calls = 0; /* number of read() calls  */
    long long write_calls = 0;/* number of write() calls */
    ssize_t n;

    /* TODO 1 (completed): read a block, then write all of it out */
    for (;;) {
        n = read(in, buf, (size_t)bufsize);
        read_calls++;
        if (n < 0) {
            if (errno == EINTR)
                continue;          /* interrupted by a signal: try again */
            perror("read");
            break;
        }
        if (n == 0)
            break;                 /* end of file */
```

```
        ssize_t done = 0;
        while (done < n) {          /* write may write less than asked */
            ssize_t w = write(out, buf + done, (size_t)(n - done));
            write_calls++;
            if (w < 0) {
                if (errno == EINTR)
                    continue;
                perror("write");
                goto finish;
            }
            done += w;
        }
        total += n;
    }

finish:
    close(in);
    close(out);
    free(buf);

    printf("Copied %lld bytes with buffer size %ld\n", total, bufsize);
    printf("read() calls:  %lld\n", read_calls);
    printf("write() calls: %lld\n", write_calls);
    return 0;
}
```

Compilation

```
gcc -O2 -Wall -o copy_rw copy_rw.c
```

Execution steps

```
head -c 1048576 /dev/urandom > test.bin      # create a 1 MB test file
./copy_rw test.bin out.bin 4096              # copy with 4 KB buffer
cmp test.bin out.bin && echo "files identical" # verify the copy
strace -c ./copy_rw test.bin out.bin 4096    # count system calls
```

Expected output

```
Copied 1048576 bytes with buffer size 4096
read() calls:  257
write() calls: 256
files identical
```

The read count is 257 because the last read returns 0 (end of file). strace -c should report the same number of read calls (257), and write calls (256). If your numbers differ, check the file size and buffer size before you explain the difference.

Measurement table

Run each buffer size at least 5 times and take the median time. Use: time ./copy_rw test.bin out.bin B and strace -c ./copy_rw test.bin out.bin B (B = buffer size).

Buffer size B	read() calls (strace -c)	write() calls (strace -c)	Real time (s)	User time (s)	Sys time (s)
1	_____	_____	_____	_____	_____
64	_____	_____	_____	_____	_____
512	_____	_____	_____	_____	_____
4096	_____	_____	_____	_____	_____

Buffer size B	read() calls (strace -c)	write() calls (strace -c)	Real time (s)	User time (s)	Sys time (s)
65536	_____	_____	_____	_____	_____
1048576	_____	_____	_____	_____	_____

Best buffer size for the hospital backup program

Use the measured table to find the knee of the curve: the point after which larger buffers save little time. Typical results show a big drop from 1 byte to 512 bytes, a smaller drop to 4 KB, and almost no change beyond 64 KB. Your recommendation should be a buffer of 64 KB (65,536 bytes) unless your measurements show a different knee. Reasons: 64 KB is small compared with RAM, it cuts calls by a factor of 16 compared with 4 KB, and it is aligned to file system blocks. Write the recommendation only after the measurements are in.

Alternative: memory-mapped files (mmap) instead of read() and write()

Measure	read() / write() with buffer	mmap()
Number of system calls	About n / b reads and n / b writes. Example: 256 each for 1 MB with 4 KB.	A few calls (open, mmap, munmap). Page faults happen on access, one per page not yet loaded: about n / 4096 faults, which are not system calls but still kernel entries.
Copying between kernel and user space	Data is copied twice (kernel buffer to user buffer, then back).	The page cache is mapped into the process; no extra copy for reading. Copy count is 0 to 1 depending on use.
Memory for very large files	Only one buffer of b bytes, O(b) space.	Mapping uses virtual address space, O(n) of virtual space but physical pages are loaded on demand. On 32-bit systems a multi-GB file may not fit in address space.

Result for Task 6. The number of system calls falls as O(n / b) when the buffer grows, while copy work stays O(n). Library buffering reduces calls to about n / 4096. The recommended backup buffer (64 KB, subject to confirmation with the measured table) balances call count and memory.

Task 7: Evaluate Process Control System Calls (4 Marks)

Bloom level: K5 Evaluate

Aim

Write a small command shell that starts programs with fork(), execlp() and waitpid(), and explain zombies, orphans and the fork-then-exec design.

What each call does

Call	What it does	Returns
fork()	Creates a copy of the calling process. The child gets a new PID and a copy of the address space (shared pages are copy-on-write).	0 in child, child PID in parent, -1 on error
execlp()	Replaces the program image of the current process with a new program. PID stays the same. The search uses PATH.	Returns only on error
waitpid()	Waits until the given child changes state (exits). Collects its exit status and frees its process table entry.	PID of child, or -1 on error
exit() / _exit()	Ends the process and gives a status to the parent. exit() flushes stdio buffers first; _exit() does not.	Does not return

How a shell runs a command and gets its exit status

The shell reads a line, calls fork(). The child calls exec to become the command. The parent calls waitpid() and waits. When the child ends, the kernel stores its status. The parent reads it with WIFEXITED() and WEXITSTATUS() and shows it to the user. The shell then prompts again.

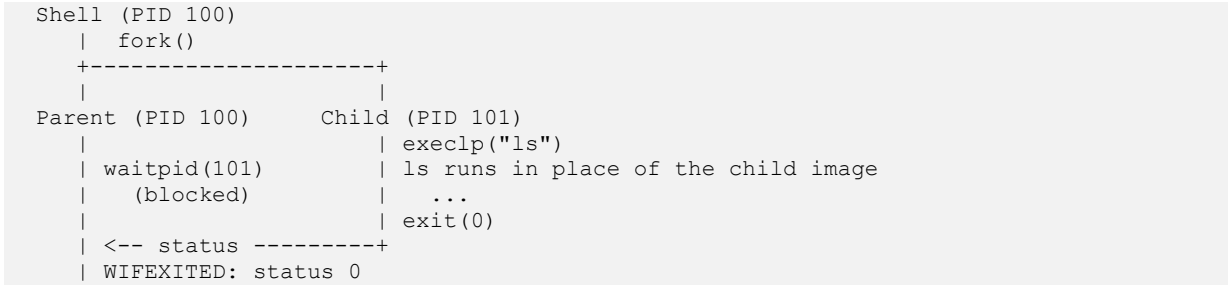
Zombie and orphan processes

- **Zombie:** a child that has exited but whose parent has not called wait. The kernel keeps a small entry (status, PID) so the parent can read the exit code. Shown as state Z in ps. It uses no CPU or memory but takes a PID. Fix: the parent calls waitpid(), or ignores SIGCHLD, or uses a double fork.
- **Orphan:** a child whose parent has exited first. The kernel gives the child to init (PID 1) or to a subreaper process, which calls wait for it. Orphans are not a problem for the system; they just have a new parent.

Why fork() followed by exec() is used instead of a single create-process call

- Between fork() and exec() the child can change its environment: redirect stdin or stdout with dup2(), close files, set limits, change directory, or set a new environment. This is how pipes and redirections work in a shell.
- It separates the "make a copy" step from the "run this program" step, so each can be simple and general.
- The cost is copying the page tables, which copy-on-write makes cheap. vfork() or posix_spawn() can avoid even that cost (see the alternative below).

Process diagram



```
     | prints "job 1 exited with status 0"
     | prompt again
```

Code: Task07/shell_fork.c (TODO 1 completed)

```
/* shell_fork.c - a tiny shell: runs one-word commands with fork/execlp/waitpid.
 * Compile: gcc -O2 -Wall -o shell_fork shell_fork.c
 * Run:     ./shell_fork
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_LINE 256

int main(void)
{
    char line[MAX_LINE];
    int job = 0;

    printf("mini-shell ready. Type a command name (for example: date), or exit.\n");

    for (;;) {
        printf("mysh> ");
        fflush(stdout);                    /* show the prompt now */

        if (fgets(line, sizeof line, stdin) == NULL)
            break;                         /* Ctrl-D ends the shell */

        line[strcspn(line, "\n")] = '\0';  /* remove the newline */
        if (line[0] == '\0')
            continue;
        if (strcmp(line, "exit") == 0)
            break;

        job++;
        printf("shell: running job %d\n", job);
        fflush(stdout);                    /* flush #1: before fork() */

        pid_t pid = fork();
        if (pid < 0) {
            perror("fork");
            continue;
        }

        if (pid == 0) {                    /* ---- child ---- */
            fflush(stdout);                /* flush #2 (child side) */
            execlp(line, line, (char *)NULL);
            perror("execlp");              /* only reached if exec failed */
            _exit(3);                      /* _exit: do not flush parent's buffers */
        }

        /* ---- parent ---- */
        int status;
        if (waitpid(pid, &status, 0) < 0) {
            perror("waitpid");
            continue;
        }
        if (WIFEXITED(status))
            printf("shell: job %d exited with status %d\n", job, WEXITSTATUS(status));
```

```
        else if (WIFSIGNALED(status))
            printf("shell: job %d killed by signal %d\n", job, WTERMSIG(status));
    }

    printf("shell: bye\n");
    return 0;
}
```

Compilation and execution

```
gcc -O2 -Wall -o shell_fork shell_fork.c
./shell_fork
```

Expected output (sample, typed input in bold in your terminal)

```
mini-shell ready. Type a command name (for example: date), or exit.
mysh> date
shell: running job 1
Thu Nov  5 10:12:44 IST 2026
shell: job 1 exited with status 0
mysh> nosuchcmd
shell: running job 2
execlp: No such file or directory
shell: job 2 exited with status 3
mysh> exit
shell: bye
```

The date and time shown are sample values. The important parts are the job numbers and the exit status: 0 for a successful command and 3 for a failed exec.

Experiment: why std out is flushed before fork()

Stdout is line buffered on a terminal but fully buffered when it is redirected to a file. Any text still in the buffer when fork() runs is copied into the child as well. If the child then calls exit(), the C library flushes that copy, so the text is written a second time. Flushing before fork() empties the buffer, so there is nothing to duplicate.

Steps for the experiment:

- Remove the flush #2 line in the child, and change _exit(3) to exit(3).
- Also remove flush #1 (the one before fork()). This is needed to make the duplicate appear, because with flush #1 in place the buffer is empty at fork time.
- Run it with output redirected: echo -e "nosuchcmd exit" | ./shell_fork > out.txt, then cat out.txt.

Result you should observe: the line "shell: running job 2" appears twice, because the failed child inherited the unflushed buffer and exit() flushed it. Note for viva: with flush #1 kept, the duplicate does not appear even with exit(), because the child starts with an empty buffer. Flush #2 in the child is redundant and only shows the same idea. A successful exec() replaces the child image, so its buffer is discarded and no duplicate is possible on that path.

Alternative: posix_spawn() or threads instead of fork() then exec()

Measure	fork() then exec()	posix_spawn()	Threads (pthread_create)
Time to start 1,000 commands	Per start: copy page tables (O(p) for p pages, reduced by copy-on-write). Total about 1,000 x fork cost.	Usually faster: glibc uses vfork-like or clone semantics, so the page table is not copied. Still O(1) per start.	Very fast (no new address space) but threads cannot exec a different program in the

Measure	fork() then exec()	posix_spawn()	Threads (pthread_create)
			same way; a thread would have to run a function.
Memory copied or shared	Copy-on-write: pages are shared until one process writes. Cost is O(pages written).	Little or no copying of page tables.	Entire address space is shared, so no copy cost.
Isolation between commands	Full. Each command has its own address space, file descriptors and crash domain.	Same as fork-exec for the new program.	Poor. A crashing thread can bring down all of them, and they share memory.

Result for Task 7. The shell uses fork, execlp and waitpid. It collects exit codes through waitpid. Zombies are removed by waitpid; orphans are adopted by init. fork then exec is used so the child can set up redirections before the new program runs. posix_spawn is a faster option with the same isolation; threads are fast but not isolated.

Task 8: Evaluate CPU States and Protection (3 Marks)

Bloom level: K5 Evaluate

Aim

Explain how the CPU hardware protects the system, and what happens to the monitor if that protection were missing.

User mode and kernel mode

- **Mode bit:** a bit in a CPU status register. 0 = kernel (privileged), 1 = user. The CPU checks it before running sensitive instructions.
- **Privileged instructions:** examples: disable interrupts, load the page table base register, halt the CPU, access I/O ports directly. Running one in user mode raises a trap, and the OS stops the program.
- **System calls:** the only legal way for a user program to ask the kernel to do privileged work (the syscall instruction, Task 5).

Interrupts, traps and the timer

Event	Cause	Example on the monitor
Hardware interrupt	A device signals the CPU	ADC finishes a sample; a new heart-rate value is ready
Trap / exception	The running program causes it	Divide by zero; illegal memory access; system call
Timer interrupt	A clock that fires at a fixed rate	Gives the scheduler a chance to switch tasks; stops a looping task from keeping the CPU

What would happen if any program could disable interrupts

- The timer could be turned off, so one task could hold the CPU forever.
- Sensor interrupts could be ignored, so a critical alarm sensor would not be served.
- Watchdog and safety tasks could be starved, so a hung device would not be reset.
- In short: the monitor would lose its guarantee of an alarm deadline. This is why the instruction is privileged.

How hardware stops one program reading another program memory

The memory management unit (MMU) translates each virtual address using the page table of the running process. The table lists only the pages that process may use, with read, write and execute flags. If a program accesses an address not in its table or without permission, the MMU raises a page-fault trap. The kernel then either loads the page (valid case) or kills the process (illegal case). Because the page table base register is privileged, a user program cannot change its own mapping.

Alternative: polling instead of interrupts on the monitor

With polling, the CPU checks the sensor in a loop every T milliseconds. With interrupts, the sensor signals the CPU when a sample is ready.

Measure	Polling every T ms	Interrupt-driven
Worst-case delay before an abnormal reading is noticed	Up to T plus processing. Example: T = 10 ms gives 10 ms worst case. Cost of checking is O(1) per check, but the number of checks is O(time / T).	Interrupt latency: typically microseconds (a few µs). Handler work is O(1) per sample.
CPU time wasted	Each check costs c microseconds even if nothing changed. Example: 1,000 checks per second at 5 µs each uses 0.5 percent CPU, all wasted when readings are steady.	Zero CPU work between samples. The CPU sleeps.
Power wasted	The CPU is awake for each check, so power stays higher.	The CPU can enter a low-power state until the next interrupt.

Worked example (illustrative): if T = 1 ms (1,000 checks per second), the worst-case delay is about 1 ms, and the CPU spends 1,000 x 5 µs = 5 ms per second, that is 0.5 percent of the CPU. Interrupts give a worst-case delay of a few microseconds and near-zero idle cost. Interrupts are chosen for the alarm path.

Result for Task 8. Dual mode, privileged instructions, the timer and MMU together protect the monitor and the records. Without privileged interrupt control, the alarm deadline could not be guaranteed. Interrupts beat polling for latency and power.

Task 9: Evaluate the Memory Hierarchy and I/O Systems (4 Marks)

Bloom level: K5 Evaluate

Aim

Place each kind of data in the right storage level, choose the right I/O method for each device, and show the effect of caching on record look-up.

Memory hierarchy (typical values)

Access times are typical orders of magnitude for current hardware. They vary by product.

Level	Typical size	Typical access time	Notes
Registers	Bytes to a few KB	About 0.3 ns (1 CPU cycle)	Fastest; managed by compiler
L1 cache	32 to 64 KB per core	About 1 ns	Split instruction and data
L2 cache	256 KB to 2 MB per core	About 3 to 4 ns	
L3 cache	8 to 64 MB shared	About 10 to 20 ns	
Main memory (DRAM)	4 GB to 1 TB	About 80 to 100 ns	Volatile
SSD (NVMe)	256 GB to tens of TB	About 20 to 100 µs	Non-volatile; random access
Hard disk (HDD)	TB range	About 5 to 10 ms	Seek time dominates random access
Network storage (LAN)	PB range	About 0.5 to 5 ms	Shared; depends on network
Cloud / WAN storage	Practically unlimited	About 50 to 200 ms	Remote; highest latency

Where data should be kept

Data	Hot working copy	Persistent copy	Archive	Reason
MRI images	Imaging server RAM during reconstruction	SSD on imaging server	Network storage (PACS-style archive)	Large (hundreds of MB each). Working set must be fast; archive is cheap and shared.
Patient records	Database buffer cache in RAM	SSD / RAID on DB cluster with replication	Encrypted backup on network storage	Lookups must be fast; integrity needs redundancy.
Monitor readings	Device RAM ring buffer (last few minutes)	Small flash on the monitor (fixed-size log)	Uploaded to database server	Readings are small and time-stamped. Local copy survives network loss.

I/O methods

Method	How it works	CPU involvement	Best for
Programmed I/O (polling)	CPU reads or writes each byte or word and checks a status register in a loop	Very high: CPU busy for the whole transfer	Very small, simple devices
Interrupt-driven I/O	Device raises an interrupt per unit (byte, word or event); CPU copies the data	One interrupt per unit; CPU copies each unit	Keyboards, sensors, low-rate devices

Method	How it works	CPU involvement	Best for
DMA (direct memory access)	DMA controller moves a block between device and memory; CPU is told once when the block is done	One interrupt per block	Large, fast transfers: disks, network cards, MRI data

- **MRI scanner:** DMA. The scanner produces a large stream of raw detector data. DMA moves whole blocks into memory with one interrupt per block, so the CPU is free to reconstruct earlier data at the same time.
- **Monitor sensors:** interrupt-driven. Each sample is small and arrives at a fixed rate, and the alarm path needs low latency.
- **Tablet touch:** interrupt-driven. Touch events are small and infrequent.

Effect of caching on record look-up

Average access time (AMAT) with a cache is: AMAT = hit rate x time on hit + miss rate x time on miss.

Worked example (illustrative): the buffer cache returns a record in 0.1 ms (RAM and index), and a disk read takes 5 ms. If 95 percent of lookups hit the cache:

```
AMAT = 0.95 x 0.1 ms + 0.05 x 5 ms
     = 0.095 ms + 0.25 ms
     = 0.345 ms
Speed-up over no cache = 5 ms / 0.345 ms = about 14.5 times faster
```

Caching works well for patient records because doctors look at the same current patients many times. Records for patients not seen recently are in the slower level, which is acceptable.

Alternative: interrupt-driven I/O instead of DMA for the MRI scanner

Assumptions (stated, not measured): image size 500 MB = 500 x 10^6 bytes; 4-byte word; block for DMA is 4 KB; interrupt cost including context switch about 2 µs per interrupt.

Measure	Interrupt per word	DMA with 4 KB blocks
Number of interrupts	500 x 10^6 / 4 = 125,000,000	500 x 10^6 / 4096 = about 122,070
CPU time spent on interrupts	125,000,000 x 2 µs = 250 s (about 4 min)	122,070 x 2 µs = about 0.24 s
Number of interrupts (formula)	O(n / w) for n bytes and word size w	O(n / B) for block size B
Memory buffers needed	One word register plus a copy target; the copy loop runs per word. Very little buffer, but a copy per word.	A ring of a few blocks (for example, 2 x 4 KB), or one large buffer for the whole image. O(B) to O(n) space, chosen by design.

Result for Task 9. Data is placed by temperature: hot working copies in RAM, persistent copies on SSD or replicated storage, archives on network storage. The MRI uses DMA because interrupt-driven transfer would spend about 250 seconds of CPU time per 500 MB image, while DMA needs about a quarter of a second under the stated assumptions

Task 10: Measure and Compare (4 Marks)

Bloom level: K5 Evaluate

Aim

Measure the copy program for different buffer sizes, measure user and kernel time with time, and measure the cost of creating 1,000 processes versus 1,000 threads. Compare the results with the earlier judgements.

Important rule for this task

The tables below are empty on purpose. Fill them only with values from your own machine. Record the machine details (CPU model, RAM, kernel version from uname -a) at the top of your report. Run each test at least 5 times and report the median. Expected trends are given so you can check whether your results support the earlier analysis.

Machine details to record

```
uname -a
lscpu | head -n 15
free -h
gcc --version | head -n 1
```

Test 1: system calls of the copy program (strace -c)

```
for b in 1 64 512 4096 65536 1048576; do
  echo "buffer=$b"
  strace -c -e trace=read,write ./copy_rw test.bin out.bin $b 2>&1 | tail -n 6
done
```

Buffer size	Expected read() count (calculated)	Measured read() count (strace -c)	Measured write() count	Expected trend
1	1,048,577	_____	_____	Very slow; huge call count
64	16,385	_____	_____	About 64 times fewer than size 1
512	2,049	_____	_____	
4096	257	_____	_____	Common default for the file system
65536	17	_____	_____	Near the knee if measured
1048576	2	_____	_____	Fewest calls; large memory use

The expected read() counts are calculated as n / b plus one. They are a check on the measurement, not the measurement itself. If strace and your program disagree, state the difference and explain it.

Test 2: user time and kernel time (time command)

```
time ./copy_rw test.bin out.bin 1
time ./copy_rw test.bin out.bin 4096
time ./copy_rw test.bin out.bin 65536
```

The time command prints real (wall clock), user (time in user mode) and sys (time in kernel mode). Small buffers should show large sys time because each call enters the kernel.

Buffer size	Real (s)	User (s)	Sys (s)	Sys share = sys / (user + sys)
1	_____	_____	_____	_____
4096	_____	_____	_____	_____
65536	_____	_____	_____	_____

Test 3: creating 1,000 processes and 1,000 threads

The program below creates and reaps 1,000 processes (fork) or 1,000 threads (pthread_create), and prints the elapsed time. Each child or thread exits at once; the parent waits for all of them, so their cleanup is included in the time.

Code: Task10/bench_create.c

```
/* bench_create.c - time creating and reaping 1000 processes or 1000 threads.
 * Compile: gcc -O2 -Wall -pthread -o bench_create bench_create.c
 * Run:     ./bench_create fork
 *          ./bench_create thread
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <time.h>

#define N 1000

static void *thread_main(void *arg)
{
    return arg;                         /* exit immediately */
}

static double now_seconds(void)
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double)ts.tv_sec + (double)ts.tv_nsec / 1e9;
}

int main(int argc, char *argv[])
{
    if (argc != 2) {
        fprintf(stderr, "Usage: %s fork|thread\n", argv[0]);
        return 1;
    }

    double t0 = now_seconds();
```

```
    if (strcmp(argv[1], "fork") == 0) {
        for (int i = 0; i < N; i++) {
            pid_t pid = fork();
            if (pid < 0) {
                perror("fork");
                return 1;
            }
            if (pid == 0)
                _exit(0);             /* child: exit at once */
        }
        for (int i = 0; i < N; i++)
            wait(NULL);               /* parent: reap all children */
    } else if (strcmp(argv[1], "thread") == 0) {
        pthread_t tid[N];
        for (int i = 0; i < N; i++) {
            if (pthread_create(&tid[i], NULL, thread_main, NULL) != 0) {
                perror("pthread_create");
                return 1;
            }
        }
        for (int i = 0; i < N; i++)
            pthread_join(tid[i], NULL);
    } else {
        fprintf(stderr, "Unknown mode: %s (use fork or thread)\n", argv[1]);
        return 1;
    }

    double t1 = now_seconds();
    printf("Created and reaped %d %ss in %.4f seconds\n", N, argv[1], t1 - t0);
    return 0;
}
```

Compilation and execution

```
gcc -O2 -Wall -pthread -o bench_create bench_create.c
for i in 1 2 3 4 5; do ./bench_create fork;   done
for i in 1 2 3 4 5; do ./bench_create thread; done
/usr/bin/time -v ./bench_create fork     2>&1 | grep -E "Maximum resident|Elapsed"
/usr/bin/time -v ./bench_create thread   2>&1 | grep -E "Maximum resident|Elapsed"
```

Expected output format (values are yours to measure)

```
Created and reaped 1000 forks in ______ seconds
Created and reaped 1000 threads in ______ seconds
```

Run	fork time (s)	thread time (s)	Max RSS fork (KB)	Max RSS thread (KB)
1	_____	_____	_____	_____
2	_____	_____	_____	_____
3	_____	_____	_____	_____
4	_____	_____	_____	_____
5	_____	_____	_____	_____
Median	_____	_____	_____	_____

Expected trend and why

Threads are usually faster to create than processes, because a new thread shares the address space, file descriptors and page tables with its creator, so the kernel only creates a new stack and task entry. A new process needs a new address space and page table copy (copy-on-write

makes the copy cheap but not free), so it costs more time and more memory. Note that the first fork test sometimes looks slower because of the first run page-cache effects; the median over 5 runs reduces that.

Comparing measurements with earlier judgements

Earlier judgement	Check with measurement	Supported or contradicted?
System calls fall as buffer grows (Task 6)	Compare strace -c counts for sizes 1 to 1,048,576	_____
Small buffers cost mostly kernel time (Task 6)	Sys share in the time table	_____
Threads are cheaper than processes (Task 7 alternative)	Compare thread and fork times	_____
Processes use more memory than threads (Task 10 alternative)	Compare Max RSS	_____

Alternative: threads instead of processes for the measurement

Measure	Creating 1,000 processes	Creating 1,000 threads
Creation time	Higher: new address space and page tables, plus PID allocation. Cost per creation is O(p) for p mapped pages without copy-on-write benefit; with it, cheaper but still more work than a thread.	Lower: O(1) bookkeeping plus a stack allocation. Stack size is set by the default thread stack (for example, 8 MB reserved, but only touched pages cost memory).
Memory used	Each process has its own copy of page tables and private memory; shared pages are counted once in the kernel but each process has its own mapping.	Threads share code, heap, global data and file descriptors. Extra memory is mainly the thread stack and thread control block.
What is shared	Only read-only or copy-on-write pages until written	Everything except stacks and registers

Result for Task 10. The results must come from your machine. The expected pattern is: buffer size controls the system-call count and the sys time; thread creation is cheaper than process creation because threads share the address space. Record each contradiction and explain it, because a contradiction is useful evidence.

Task 11: Recommend and Defend the Operating-System Plan

Bloom level: K5 Evaluate

Aim

Give one complete plan for the hospital, based on the analysis in Tasks 1 to 10 and the measurements.

Recommended plan

Environment	OS type	Structure	Main services	Protection	I/O method
E1 Bedside monitor	Hard real-time	Microkernel (QNX-style) or compact RTOS	Sensor I/O, alarm, watchdog, logging to flash	Signed firmware, task isolation, mode bit	Interrupt-driven
E2 Nurse tablet	Handheld	Layered (Android-style, reduced services)	Touch UI, network, encrypted storage, battery management	App sandbox, device encryption, remote wipe	Interrupt-driven, DMA for storage
E3 Workstation	Desktop, time-sharing	Modular monolithic (Linux with modules)	GUI, file system, printing, audit	Login, file ACLs, audit logs	DMA for disks and network
E4 DB server	Clustered with fail-over on multiprocessor nodes	Modular monolithic (Linux)	Storage, transaction services, replication, accounting	Record ACLs, encryption, audit	DMA; buffer cache
E5 Imaging server	Multiprocessor, symmetric (SMP)	Modular monolithic (Linux)	Parallel jobs, GPU driver, storage	Access control, encrypted archive	DMA
E6 Partner link	Hardened embedded Linux appliance	Modular monolithic, minimal services	Firewall, VPN, retry logic	mTLS, firewall rules, signed updates	DMA for network
E7 MRI controller	Hard real-time	Microkernel or RTOS	Timing, data acquisition	Isolated control network	DMA (data), interrupt (control)

Limitations and risks

- **Real-time limits:** a microkernel or RTOS has less software support than Linux, so drivers must be written and tested carefully.
- **Mixed OS types:** more than one OS to patch and monitor. Mitigation: central inventory and staged updates.
- **Fail-over risk:** split-brain can corrupt records if fencing fails. Mitigation: test fencing monthly.
- **Tablet battery:** low battery can stop nurses receiving alerts. Mitigation: alerts on low battery.
- **Network dependency:** partner link and any remote connection can fail. Monitors and local records must keep working without it.
- **Human error:** wrong access rights or wrong patient selection. Mitigation: role-based access and two-step checks in the record system.

Suitability for a group of three hospitals

The plan scales if each hospital has its own monitors and workstations, and the database and partner link are shared through a secure, replicated service. It needs: consistent naming and access rules, a common audit format, data-sharing agreements, and an independent check of time synchronisation across sites. Limits: each hospital keeps its own safety-critical devices, so a shared failure in the central service must not stop bedside care.

Alternative: virtual machines on two large servers

Measure	Physical machines per environment	Virtual machines on two servers
Response time for monitors	Lowest and most predictable: interrupts go straight to the device.	Higher and less predictable: hypervisor adds interrupt delay. Hard deadlines are difficult to guarantee with shared hosts.
Memory and CPU overhead	None beyond each OS	Hypervisor and per-VM kernel use memory and some CPU; typically a few percent of CPU, plus memory per VM. Overhead is O(v) for v VMs.
Reliability	Failure of one device affects only that device	Failure of one large server can stop many VMs at once unless live migration and fail-over are set up.

Result for Task 11. The plan uses hard real-time systems for monitors and MRI control, a multiprocessor SMP system for imaging, a clustered fail-over system for records, and desktop and handheld systems where interaction matters. Virtualisation is useful for the database and workstations, but monitors should stay on physical machines for their deadlines.

Task 12: Re-evaluate and Adapt the Design

Bloom level: K5 Evaluate

Changed requirement

The hospital adds remote monitoring devices in patients homes that send readings over the internet, and moves patient records to a cloud service shared with partner hospitals. The decision for this task is to RETAIN, MODIFY or REDESIGN the Task 11 plan. The answer is to MODIFY it, and to REDESIGN only the new parts (home devices and cloud records).

Part A: Impact analysis

Area	Original design (Task 11)	What changes	Impact level
OS types	Hard real-time (monitors), SMP (imaging), clustered (DB), desktop, handheld	New: embedded RTOS on home devices; distributed system for shared records; internet-facing gateway	High
OS structures	Microkernel/RTOS on monitors; modular Linux on servers	Home devices need small, updatable firmware with secure boot; cloud needs containers and service isolation	High
Services	Local storage, local network, local audit	Need TLS, retry queues, time synchronisation over WAN, data residency and consent tracking	High
Protection	Physical access control, VLANs, local ACLs	Remote attackers, stolen home devices, cloud tenancy, shared records between hospitals	Very high
Response time	Local, microseconds to milliseconds	Home-to-hospital path adds 50 to 200 ms WAN delay, plus jitter. Home alarms must still be local first	High for remote alarms
Reliability	Local failover	Internet outages, cloud region outage, device offline. Needs store-and-forward	High

Part B: Evaluation of the original design

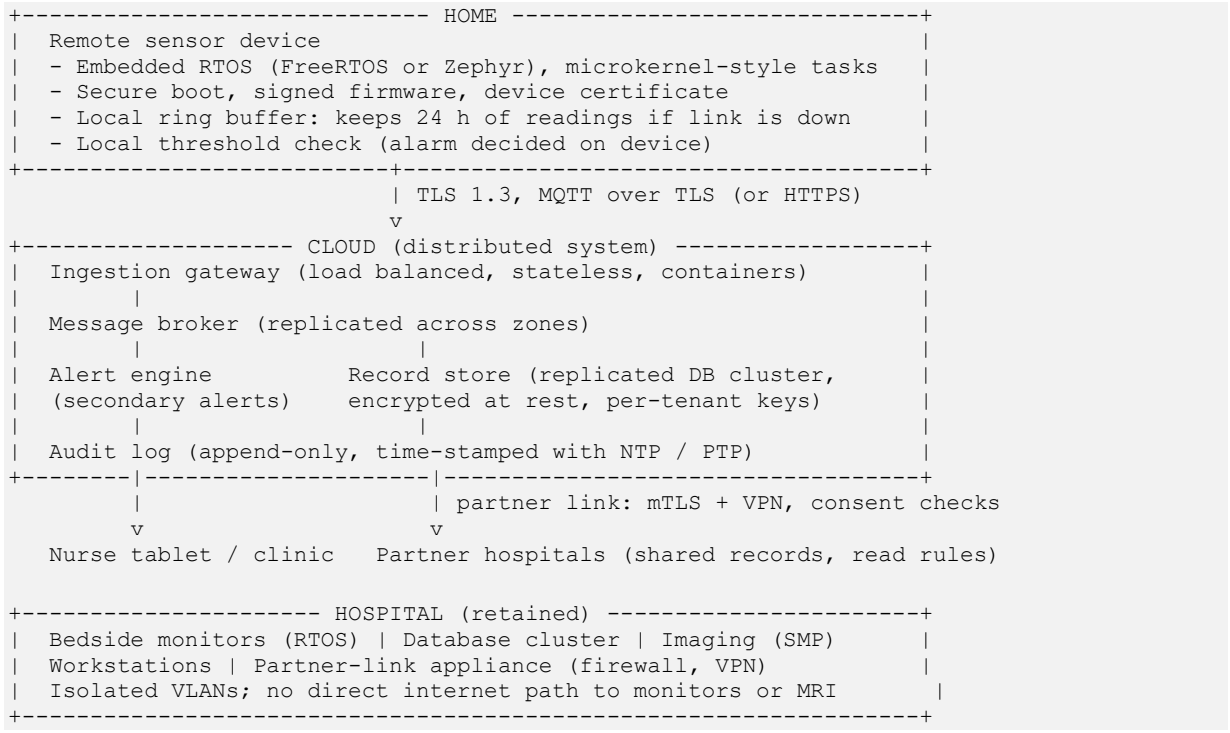
The Task 11 design cannot support home devices and cloud records as it stands, for three reasons.

- **Response time:** the monitor and alarm logic were designed for a local network. Over the internet, a reading may arrive 50 to 200 ms late or not at all. A remote alarm based on a WAN path cannot meet the bedside deadline. Decision: alarm decisions stay on the device; the cloud is for records and secondary alerts.
- **Reliability:** the fail-over cluster is local. A cloud service needs its own replication across zones or sites, and the home device needs a local buffer so readings are not lost during an outage.
- **Security:** the Task 11 plan trusts the hospital network. Home devices sit outside it and can be physically stolen. The original OS lacks secure boot, device identity and remote update signing for that case.

What still holds: the monitor design, the local database cluster, and the imaging design remain valid for in-hospital use.

Part C: Adaptation or redesign

Revised plan: MODIFY the hospital design and REDESIGN the two new areas. The architecture is shown below.



Key design decisions

- **Alarm locality:** the home device decides the threshold alarm itself and sends the event. The cloud never has to be in the alarm loop. This keeps the deadline logic of Task 1.
- **Store and forward:** readings are kept on the device and sent with sequence numbers, so the cloud can remove duplicates after an outage.

- **Distributed records:** the record store is a replicated cluster across availability zones. Sharing with partners uses role-based rules and consent tags, not raw copies.
- **Secure boot and updates:** home devices verify signed firmware. Updates are staged (canary group first) and can roll back.
- **Zero trust:** every connection, including inside the cloud, uses mutual TLS with device or service identity. No implicit trust by network location.
- **Time:** all readings are time-stamped at source and synchronised with NTP, so records from different sites can be ordered.
- **Compliance:** data residency and consent must follow the applicable local data-protection law and the partner agreements. This needs legal review before go-live.

Part D: Evidence-based defence

Evidence used:

- Task 1 ranking: deadline and reliability are above throughput, so alarm logic stays local (critical requirement).
- Task 3 structure evidence: a microkernel or RTOS isolates faults, so a faulty network driver on a home device restarts the driver, not the whole device.
- Task 6 and Task 9 evidence: batching uploads (for example, 64 KB or one message per minute) reduces calls and network overhead, while storing locally with DMA avoids CPU cost.
- Task 8 evidence: interrupts for sensor input keep idle power low on battery-powered home devices. Polling would waste power.
- Task 10 measurements (your own): use your copy and process measurements to justify batch size and thread-versus-process choice for the cloud ingestion service.

Trade-offs:

Option	Benefit	Cost or risk	Decision
Full cloud alarm logic	Central control, easy updates	WAN delay and outage can stop alarms	Rejected for threshold alarms
Alarm on device, records in cloud	Deadline kept; records shared	Device needs secure boot and local storage	Chosen
Keep records only on hospital servers	Simple, local control	Partners cannot share easily; no home access	Rejected
Replicated cloud record store	High availability, sharing	Cost, legal review, residency	Chosen with legal check
Single cloud region	Lower cost	Region outage stops records	Rejected

Final decision. MODIFY the hospital plan: keep the Task 11 OS choices for in-hospital systems. REDESIGN the home devices as an embedded RTOS with secure boot and store-and-forward, and REDESIGN the shared records as a distributed, replicated, encrypted cloud store with mutual TLS. The threshold alarm stays on the device so that the Task 1 deadline is kept. The main risks are WAN delay for secondary alerts, cost and legal compliance, and each must be checked before go-live.